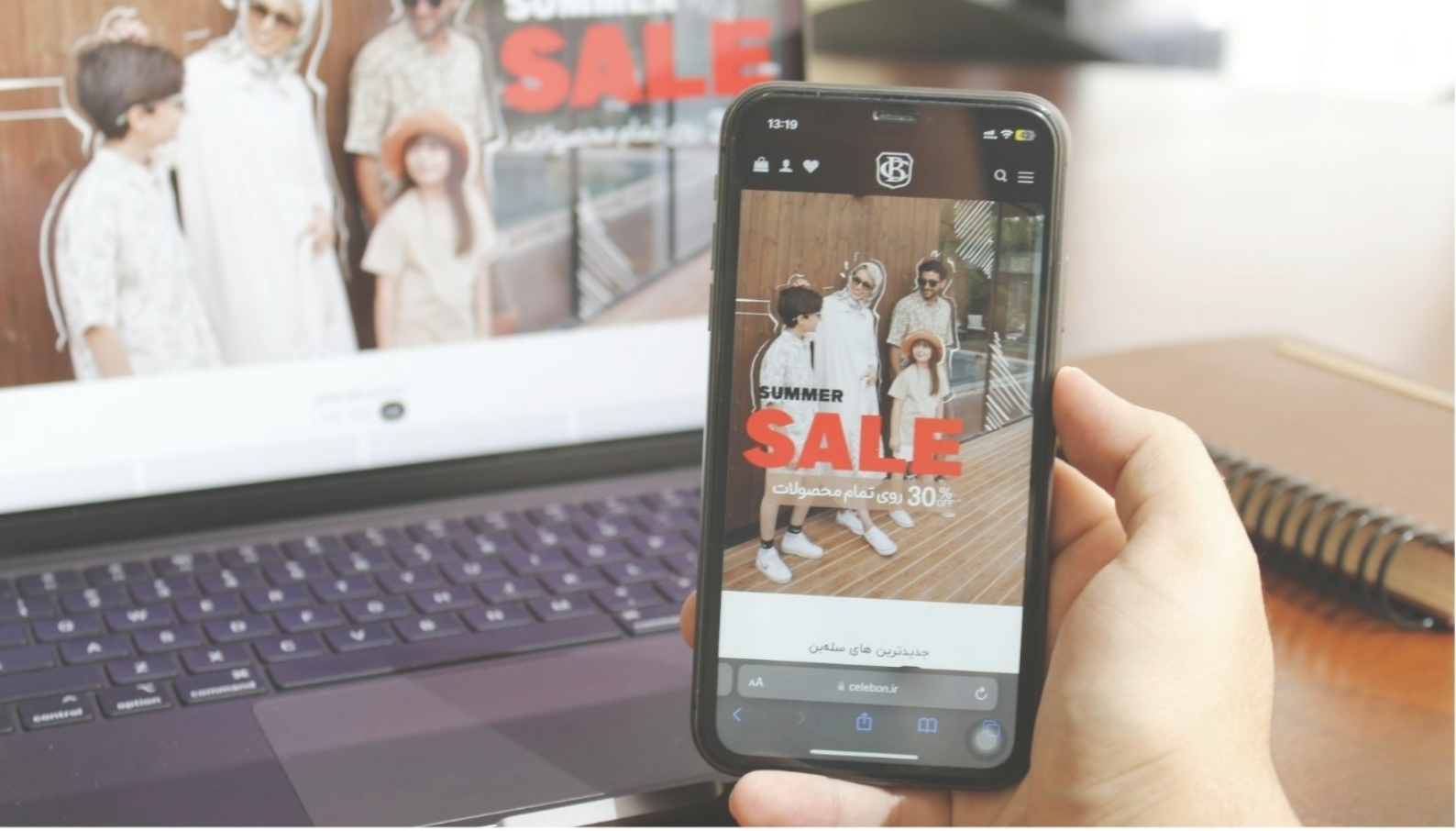


UT 1 FUNDAMENTOS DE FLUTTER

En esta unidad de trabajo aprenderás qué es una interfaz gráfica y los fundamentos básicos de Flutter, un framework para crear interfaces de aplicaciones.

Índice

1.1 SESIÓN 1.....1
1.2 Introducción a Flutter.....1
1.2.1 Desarrollo nativo vs multiplataforma.....1
1.2.2 React Native y Flutter.....2
1.2.3 Flutter y React Native: dos alternativas válidas.....3
1.3 ¿Por qué utilizaremos Flutter?.....3
1.4 Una característica importante: Flutter es declarativo.....4
1.5 En Flutter casi todo es un widget.....6
1.6 ¿Qué hemos aprendido?.....7
1.7 SESIÓN 2.....8
1.8 Anatomía de una aplicación Flutter.....8
1.8.1 El punto de partida: main().....9
1.8.2 runApp(): ponemos en marcha Flutter.....9
1.8.3 MaterialApp: la aplicación.....10
1.8.4 Scaffold: la estructura de una pantalla.....10
1.8.5 Siguiendo el código hasta llegar a la interfaz.....11
1.8.6 Añadimos una barra superior.....12
1.9 La interfaz y el código están relacionados.....13
1.10 Lo que hemos aprendido.....13
1.11 SESIÓN 3.....14
1.12 WIDGETS BÁSICOS Y COMPOSICIÓN DE INTERFACES.....14
1.12.1 child y children.....14
1.12.1.1 child: un único widget hijo.....14
1.12.1.2 children: varios widgets hijos.....15
1.13 Organizamos widgets con Column y Row.....15
1.13.1 Column: organización vertical.....15
1.13.2 Row: organización horizontal.....16
1.14 Podemos anidar widgets.....16

1.15 Algunos widgets básicos....17
1.16 ListTile: información organizada en una fila....20
1.17 Construir una interfaz: primero pensamos en su estructura....21
1.18 Lo que hemos aprendido....22
1.19 SESIÓN 4....23
1.20 Widgets básicos y composición de interfaces....23
1.20.1 Construimos interfaces combinando widgets....23
1.20.1.1 child: un único widget hijo....23
1.20.1.2 children: varios widgets hijos....24
1.20.1.3 Column: colocar widgets verticalmente....24
1.20.1.4 Row: colocar widgets horizontalmente....25
1.20.1.5 Podemos anidar widgets....25
1.20.2 Cinco widgets básicos que vamos a utilizar....26
1.20.3 Cómo se usan....27
1.21 SESIÓN 5....27
1.22 Layout y distribución de widgets....27
1.22.1 Row y Column: dos direcciones....28
1.23 mainAxisAlignment: distribuir por el eje principal....29
1.24 crossAxisAlignment: alinear por el eje transversal....30
1.25 ¿Por qué algunos widgets tienen child, otros children y otros ninguno?....31
1.26 Center: centrar un widget....32
1.27 3.6. Padding: dejar espacio alrededor....33
1.28 SizedBox: tamaño y separación....34
1.29 3.8. Expanded: ocupa el espacio disponible....35
1.30 Spacer: espacio flexible....36
1.31 Stack: colocar widgets unos encima de otros....37
1.32 Positioned: colocar un elemento dentro de un Stack....38
1.33 Wrap: cuando los elementos no caben....39
1.34 Container: un widget muy versátil....39
1.35 Construir un layout: pensar antes de programar....40
1.36 Flutter Inspector: ver el árbol de widgets....41
1.36.1 Una idea importante sobre el layout de Flutter....42
1.37 SESIÓN 6....42
1.38 Propiedades y personalización de widgets....42
1.38.1 Propiedades más utilizadas....43
1.38.2 Texto....43
1.38.3 Iconos....43
1.38.4 Imágenes....43

1.38.5 Botones....44
1.38.6 Card....44
1.38.7 Container y BoxDecoration....44
1.38.8 ClipRRect....45
1.38.9 Colores....45
1.38.10 ¿Qué propiedad necesito?....45
1.39 SESIÓN 7....46
1.40 Eventos y callbacks....46
1.40.1 onPressed....46
1.40.2 onChanged....47
1.40.2.1 ¿De dónde sale texto?....47
1.40.3 onSubmitted....47
1.40.4 onTap....48
1.40.5 Con parámetros / sin parámetros....48
1.41 Gestión del estado....49
1.41.1 StatelessWidget....50
1.41.2 ¿Qué significa cada parte?....50
1.41.3 ¿Qué ocurre si la interfaz tiene que cambiar?....51
1.41.4 Estado....51
1.41.5 StatefulWidget....52
1.41.6 ¿Para qué sirve cada una?....52
1.41.7 setState()....53
1.41.8 Ejemplo completo....54
1.41.9 Otros ejemplos de estado....55
1.41.10 ¿StatelessWidget o StatefulWidget?....56
1.42 SESIÓN 8....56
1.42.1 Navegación entre pantallas....56
1.42.2 ¿Qué es Navigator?....57
1.42.2.1 La pila de navegación....57
1.42.2.2 Abrir una pantalla con Navigator.push()....58
1.42.2.3 Regresar con Navigator.pop()....60
1.42.2.4 Abrir con Navigator.pushReplacement()....61
1.42.2.5 Regresar con Navigator.popuntil()....61
1.42.2.6 Ejemplo navegación entre dos pantallas....62

1.1 SESIÓN 1

1.2 INTRODUCCIÓN A FLUTTER

Hasta ahora hemos programado aplicaciones utilizando distintos lenguajes, pero en este módulo nuestro objetivo será diferente: **aprender a construir interfaces gráficas de usuario**.

Una interfaz gráfica de usuario o **GUI (Graphical User Interface)** es la parte visual de una aplicación con la que interactúa el usuario: pantallas, textos, imágenes, botones, campos de entrada, menús, listas, etc.

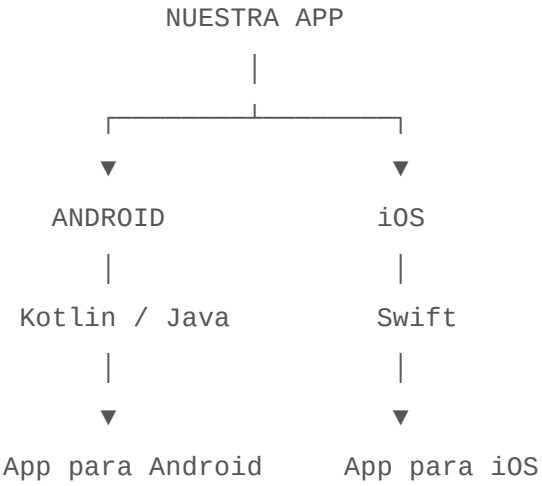
Antes de comenzar a construirlas debemos tomar una primera decisión:

> **¿Qué tecnología vamos a utilizar para desarrollar nuestras interfaces?**

1.2.1 DESARROLLO NATIVO VS MULTIPLATAFORMA

Imaginemos que queremos desarrollar una aplicación móvil que debe funcionar tanto en Android como en iOS.

Una posibilidad consiste en desarrollar una aplicación específica para cada plataforma.

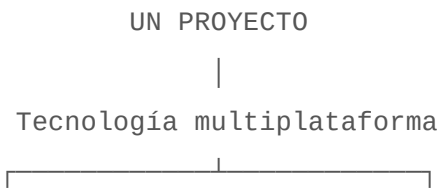


Es lo que denominamos **desarrollo nativo**.

Cada aplicación se desarrolla utilizando las tecnologías propias de la plataforma.

Este enfoque permite aprovechar directamente las características de cada sistema operativo, pero presenta un inconveniente evidente: si queremos mantener una aplicación para varias plataformas, tendremos que desarrollar y mantener código específico para cada una de ellas.

Otra posibilidad es utilizar una tecnología que nos permita compartir gran parte del código entre las diferentes plataformas.





A esto lo denominamos **desarrollo multiplataforma**.

Existen distintas tecnologías para hacerlo. Dos de las más conocidas son: **React Native** y **Flutter**.

1.2.2 REACT NATIVE Y FLUTTER

React Native

React Native es un framework de código abierto que permite desarrollar aplicaciones utilizando **React** y **JavaScript o TypeScript**.

Su interfaz se construye mediante **componentes**.

Por ejemplo:

```
<View>
  <Text>Hola</Text>
  <Button title="Aceptar" />
</View>
```

Podemos representar la estructura anterior así:

```
View
|
├── Text
|     "Hola"
|
└── Button
      "Aceptar"
```

Los componentes pueden contener otros componentes, por lo que terminamos construyendo una estructura jerárquica.

Flutter

Flutter es un **SDK de desarrollo multiplataforma** que permite construir aplicaciones para distintas plataformas a partir de una misma base de código.

Flutter utiliza el lenguaje de programación **Dart**.

En Flutter las interfaces se construyen mediante **widgets**.

Por ejemplo:

```
Column(
  children: [
    Text('Hola'),
    ElevatedButton(
      onPressed: () {},
      child: Text('Aceptar'),
    ),
  ],
)
```

Su estructura puede representarse así:

```
Column
│
├── Text
│      "Hola"
│
└── ElevatedButton
       │
       └── Text
             "Aceptar"
```

Esta estructura recibe el nombre de **árbol de widgets** (*Widget Tree*).

> **IDEA IMPORTANTE**
>
> En Flutter construimos las interfaces combinando widgets.
> Un widget puede contener otros widgets y, de esta forma, vamos formando un árbol.

1.2.3 FLUTTER Y REACT NATIVE: DOS ALTERNATIVAS VÁLIDAS

React Native y Flutter persiguen objetivos similares, pero utilizan tecnologías diferentes.

Característica	Flutter	React Native
Lenguaje principal	Dart	JavaScript / TypeScript
Construcción de la UI	Widgets	Componentes
Modelo de interfaz	Declarativo	Declarativo
Android	Sí	Sí
iOS	Sí	Sí
Hot Reload / actualización rápida durante el desarrollo	Sí	Sí
Creación de componentes propios	Sí	Sí

Por tanto, **React Native no es una alternativa "peor" a Flutter**. La elección depende del proyecto, del equipo de desarrollo y de las tecnologías que ya conozca.

Por ejemplo, para una empresa con un equipo experimentado en React y TypeScript, React Native puede resultar una elección muy razonable.

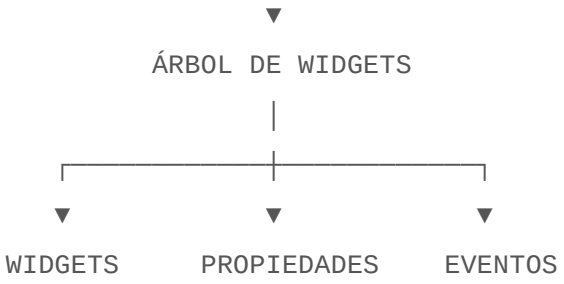
1.3 ¿POR QUÉ UTILIZAREMOS FLUTTER?

En este módulo utilizaremos **Flutter**.

Una de las razones es que el objetivo fundamental de Desarrollo de Interfaces no consiste simplemente en aprender una tecnología concreta, sino en comprender cómo se **construye, organiza y modifica una interfaz gráfica**.

Flutter permite observar claramente esa construcción:

```
INTERFAZ
   │
```



A lo largo del módulo aprenderemos progresivamente a:

```
utilizar widgets
        ↓
organizarlos
        ↓
modificar sus propiedades
        ↓
responder a eventos
        ↓
gestionar el estado
        ↓
crear nuestros propios widgets
        ↓
construir aplicaciones completas
```

Además, Flutter nos permitirá trabajar posteriormente con características propias de las aplicaciones actuales, como sensores, cámara, localización, voz y otras formas de interacción.

Por tanto, nuestra elección no significa:

> ❌ «Flutter es mejor que React Native».

Significa:

> ✓ **Flutter resulta especialmente adecuado para los objetivos que vamos a trabajar en este módulo.**

1.4 UNA CARACTERÍSTICA IMPORTANTE: FLUTTER ES DECLARATIVO

Flutter utiliza un **modelo declarativo para construir la interfaz**.

Pero ¿qué significa esto?

Vamos a imaginar una aplicación que dispone de modo claro y modo oscuro.

Tenemos una variable:

```
bool modoOscuro = false;
```

Si vale `false`, queremos:

```
┌──────────────────────────────────┐
│                                  │
│        MI APLICACIÓN             │
│                                  │
│        Hola, usuario             │
│                                  │
│        [ MODO OSCURO ]           │
│                                  │
└──────────────────────────────────┘
```

`Fondo claro`

`Texto oscuro`

El usuario pulsa el botón y cambia el estado:

`modoOscuro = true;`

Ahora queremos:

```
┌──────────────────────────────────┐
│                                  │
│        MI APLICACIÓN             │
│                                  │
│        Hola, usuario             │
│                                  │
│        [ MODO CLARO ]            │
│                                  │
└──────────────────────────────────┘
```

`Fondo oscuro`

`Texto claro`

En un modelo **imperativo**, el programa podría indicar las modificaciones que deben realizarse sobre los elementos existentes:

```
Cambia el fondo a oscuro
        ↓
Cambia el título a blanco
        ↓
Cambia el texto a blanco
        ↓
Cambia el aspecto del botón
```

```
        ↓
Cambia el texto del botón
```

Estamos indicando **qué cambios debe hacer el programa**.

Flutter utiliza otra forma de pensar.

Describimos cómo debe ser la interfaz para el estado actual:

```
              ESTADO

         modoOscuro = false
                 |
                 ▼
          INTERFAZ CLARA
```

```
         modoOscuro = true
                 |
                 ▼
          INTERFAZ OSCURA
```

Nosotros modificamos el **estado** y Flutter se encarga de actualizar la interfaz para que corresponda con ese nuevo estado.

Podemos resumirlo así:

> **Imperativo → indicamos qué cambios hay que realizar.**
>
> **Declarativo → describimos cómo queremos que sea la interfaz para el estado actual.**

Una forma habitual de expresar esta idea es:

```
           ESTADO
              |
              ▼
      ┌──────────────┐
      |  INTERFAZ    |
      |  = f(estado) |
      └──────────────┘
```

La interfaz que vemos depende del estado actual de la aplicación.

React Native también utiliza un modelo declarativo, por lo que esta característica **no es exclusiva de Flutter**.

1.5 EN FLUTTER CASI TODO ES UN WIDGET

Los **widgets son los bloques con los que construimos una interfaz Flutter**.

Algunos representan elementos visibles:

`Text` → `texto`

```
Image                → imagen

Icon                 → icono

ElevatedButton       → botón
```

Pero otros widgets sirven para **organizar** los elementos.

Dos de los primeros que utilizaremos son:

Column

Organiza sus elementos verticalmente:

```
Column
│
├── 👤
│
├── Laura Pérez
│
├── laura@email.com
│
└── 2.º DAM
```

Row

Organiza sus elementos horizontalmente:

```
Row

[ EDITAR ]    [ SALIR ]
```

Podemos combinar ambos.

```
Column
│
├── Icon
│
├── Text
├── Text
│
└── Row
     │
     ├── ElevatedButton
     │
     └── ElevatedButton
```

Y así empezamos a construir nuestro **árbol de widgets**.

1.6 ¿QUÉ HEMOS APRENDIDO?

Al terminar esta primera sesión deberías ser capaz de explicar estas ideas:

GUI

Es la parte gráfica de una aplicación mediante la que interactuamos con ella.

Desarrollo multiplataforma

Permite desarrollar aplicaciones destinadas a distintas plataformas compartiendo una parte importante de su código.

Flutter

Es el SDK multiplataforma que utilizaremos para desarrollar las interfaces del módulo.

React Native

Es otra tecnología multiplataforma válida. Utiliza React y JavaScript/TypeScript.

Modelo declarativo

Describimos cómo debe ser la interfaz para el estado actual de la aplicación, en lugar de programar individualmente cada modificación que debe sufrir la interfaz.

Widget

Es uno de los bloques con los que construimos una interfaz Flutter.

Árbol de widgets

Es la estructura jerárquica que forman los widgets al contener unos a otros.

Y, sobre todo, quédate hoy con esta idea:

```
INTERFAZ VISUAL
       ↕
ÁRBOL DE WIDGETS
       ↕
  CÓDIGO DART
```

Durante las próximas sesiones aprenderemos a pasar progresivamente de una representación a otra.

1.7 SESIÓN 2

1.8 ANATOMÍA DE UNA APLICACIÓN FLUTTER

Hasta ahora hemos visto qué es Flutter, hemos trabajado con **widgets** y hemos realizado nuestros primeros cambios sobre una interfaz.

Pero cuando ejecutamos una aplicación como esta:

```
import 'package:flutter/material.dart';

void main() {
  runApp(
    MaterialApp(
```

```
      home: Scaffold(
        body: Center(
          child: Text('Hola Flutter'),
        ),
      ),
    ),
  );
}
```

aparecen varios elementos que todavía no hemos estudiado:

`main()`, `runApp()`, `MaterialApp`, `Scaffold`...

Vamos a descubrir **qué función cumple cada uno** y cómo se relaciona el código con lo que vemos en pantalla.

1.8.1 EL PUNTO DE PARTIDA: `MAIN()`

Todo programa necesita un lugar por el que comenzar.

En Dart, ese lugar es la función:

```
void main() {
  ...
}
```

Cuando ejecutamos nuestra aplicación, Dart busca `main()` y comienza a ejecutar las instrucciones que contiene.

Podemos imaginarlo así:

```
Ejecutamos el programa
        |
        ▼
      main()
        |
        ▼
 comienza la aplicación
```

> 💡 **`main()` pertenece a Dart**, no a Flutter. También aparece en programas Dart que no tienen ninguna interfaz gráfica.

1.8.2 `RUNAPP()`: PONEMOS EN MARCHA FLUTTER

Dentro de `main()` encontramos:

```
runApp(
  MaterialApp(...)
);
```

`runApp()` es una función de Flutter que recibe un **widget** y lo coloca como punto de partida de nuestra interfaz.

En nuestro ejemplo, ese widget es `MaterialApp`:

```
main()
  |
  └── runApp()
         |
         └── MaterialApp
```

Por tanto, podemos leer:

```
runApp(
  MaterialApp(...)
);
```

como:

> **Inicia la aplicación utilizando `MaterialApp` como widget raíz.**

1.8.3 MATERIALAPP: LA APLICACIÓN

Observa ahora esta parte:

```
MaterialApp(
  home: Scaffold(...)
)
```

`MaterialApp` es un widget que configura una aplicación basada en **Material Design**.

Más adelante podremos utilizarlo para configurar aspectos generales de la aplicación, pero de momento nos interesa una propiedad:

```
home:
```

`home` indica cuál será la **pantalla inicial** de nuestra aplicación.

Por ejemplo:

```
MaterialApp(
  home: Scaffold(...)
)
```

indica que al arrancar la aplicación se mostrará el `Scaffold` que hemos colocado en `home`.

1.8.4 SCAFFOLD: LA ESTRUCTURA DE UNA PANTALLA

Una aplicación móvil suele tener zonas habituales: una barra superior, una zona principal de contenido, botones, etc.

Flutter proporciona el widget `Scaffold` para crear esa **estructura básica de una pantalla**.

Por ejemplo, un `Scaffold` podría contener:

```
┌──────────────────────────────┐
|            AppBar           |
```

```
┌──────────────────────────────┐
│                              │
│                              │
│                              │
│            body              │
│                              │
│                              │
│                        [ + ]│
└──────────────────────────────┘
```

Entre otras posibilidades, un `Scaffold` puede disponer de:

- `appBar` → barra superior.
- `body` → contenido principal.
- `floatingActionButton` → botón flotante.

En nuestra primera aplicación solamente estamos utilizando:

```
Scaffold(
  body: ...
)
```

Por tanto, todo lo que coloquemos en `body` aparecerá en la **zona principal de la pantalla**.

1.8.5 SIGUIENDO EL CÓDIGO HASTA LLEGAR A LA INTERFAZ

Volvamos ahora al código completo:

```
import 'package:flutter/material.dart';

void main() {
  runApp(
    MaterialApp(
      home: Scaffold(
        body: Center(
          child: Text('Hola Flutter'),
        ),
      ),
    ),
  );
}
```

Podemos seguirlo desde el inicio:

```
main()
  │
  └── runApp()
        │
        └── MaterialApp
              │
              └── Scaffold
                    │
                    └── Center
```

```
                          │
                          └── Text
                              "Hola Flutter"
```

Y finalmente en la pantalla vemos:

```
┌──────────────────────────────┐
│                              │
│                              │
│         Hola Flutter         │
│                              │
│                              │
└──────────────────────────────┘
```

Esto nos muestra una característica fundamental de Flutter:

> **Existe una relación directa entre la estructura del código y la estructura de la interfaz.**

La interfaz se construye combinando widgets, y esos widgets forman una **jerarquía**.

En las próximas horas aprenderemos a interpretar y construir esa jerarquía con más detalle.

1.8.6 AÑADIMOS UNA BARRA SUPERIOR

Hasta ahora nuestro `Scaffold` solo tiene `body`.

Podemos añadir una segunda zona:

```
Scaffold(
  appBar: AppBar(
    title: Text('Mi primera app'),
  ),
  body: Center(
    child: Text('Hola Flutter'),
  ),
)
```

Ahora podemos relacionar código e interfaz:

```
Scaffold
│
├── appBar ──────────────────► Mi primera app
│
└── body ────────────────────► Hola Flutter
```

El resultado será aproximadamente:

```
┌──────────────────────────────┐
│       Mi primera app         │
├──────────────────────────────┤
│                              │
│         Hola Flutter         │
```

```
│                              │
└──────────────────────────────┘
```

Observa algo importante: **no hemos dibujado una barra**. Hemos indicado que nuestro `Scaffold` tiene un `AppBar` y Flutter se ha encargado de construirla.

1.9 LA INTERFAZ Y EL CÓDIGO ESTÁN RELACIONADOS

Ya podemos interpretar una aplicación Flutter sencilla en dos direcciones.

Si vemos este código:

```
Scaffold(
  appBar: AppBar(
    title: Text('Mi aplicación'),
  ),
  body: Center(
    child: Text('Bienvenido'),
  ),
)
```

podemos imaginar aproximadamente su interfaz.

Y si vemos:

```
┌──────────────────────────────┐
│         Mi aplicación        │
├──────────────────────────────┤
│                              │
│          Bienvenido          │
│                              │
└──────────────────────────────┘
```

podemos empezar a pensar qué widgets podrían haberla construido.

Esta relación será fundamental durante todo el módulo:

INTERFAZ VISUAL ↔ ESTRUCTURA DE WIDGETS ↔ CÓDIGO DART

1.10 LO QUE HEMOS APRENDIDO

Al terminar estas dos horas debemos poder reconocer la estructura básica:

```
main()
  │
  └── runApp()
        │
        └── MaterialApp
              │
              └── Scaffold
                    ├── appBar
                    └── body
```

Y recordar:

Elemento	Función
`main()`	Punto de entrada del programa Dart
`runApp()`	Inicia la interfaz Flutter
`MaterialApp`	Configura la aplicación Material
`home`	Establece la pantalla inicial
`Scaffold`	Proporciona la estructura básica de una pantalla
`appBar`	Barra superior de la pantalla
`body`	Contenido principal de la pantalla

Idea clave

> **En Flutter, lo que vemos en pantalla es el resultado de una estructura de widgets definida mediante código.**

1.11 SESIÓN 3

1.12 WIDGETS BÁSICOS Y COMPOSICIÓN DE INTERFACES

En la sesión anterior aprendimos a reconocer la estructura básica de una aplicación Flutter. Ahora vamos a centrarnos en los elementos con los que construimos su interfaz.

En Flutter, una interfaz se construye **combinando widgets**. Algunos widgets muestran contenido, otros permiten realizar acciones y otros nos ayudan a agrupar y organizar los elementos.

Durante esta sesión aprenderemos a seleccionar los widgets adecuados y a combinarlos formando estructuras cada vez más complejas.

1.12.1 `CHILD` Y `CHILDREN`

Flutter utiliza principalmente las propiedades `child` y `children` para indicar qué widgets contiene otro widget.

1.12.1.1 `CHILD`: UN ÚNICO WIDGET HIJO

Algunos widgets contienen **un único widget**. En estos casos utilizamos `child`.

Por ejemplo:

```
ElevatedButton(
```

```
  onPressed: () {},
  child: Text('ACEPTAR'),
)
```

Su estructura es:

```
ElevatedButton
|
└── Text
```

`ElevatedButton` tiene un único hijo: `Text`.

Por tanto:

```
child → UN widget hijo
```

1.12.1.2 `CHILDREN`: VARIOS WIDGETS HIJOS

Otros widgets pueden contener **varios widgets**. En estos casos utilizamos `children`.

```
Column(
  children: [
    Icon(Icons.person),
    Text('Harry Potter'),
    Text('harry@email.com'),
  ],
)
```

Los corchetes `[ ]` indican que estamos proporcionando una **lista de widgets**.

Su estructura es:

```
Column
├── Icon
├── Text
└── Text
```

Por tanto:

```
child       → UN widget hijo

children    → VARIOS widgets hijos
```

> **Importante: debemos fijarnos siempre en los hijos directos de un widget.**

1.13 ORGANIZAMOS WIDGETS CON `COLUMN` Y `ROW`

1.13.1 `COLUMN`: ORGANIZACIÓN VERTICAL

`Column` organiza sus hijos **verticalmente**, uno debajo de otro.

```
Column(
  children: [
    Icon(Icons.person),
    Text('Harry Potter'),
    Text('harry@email.com'),
  ],
)
```

Visualmente:

```
      
   Harry Potter
harry@email.com
```

Estructuralmente:

```
Column
├── Icon
├── Text
└── Text
```

1.13.2 ROW: ORGANIZACIÓN HORIZONTAL

Row organiza sus hijos **horizontalmente**, uno al lado del otro.

```
Row(
  children: [
    Icon(Icons.email),
    Text('harry@email.com'),
  ],
)
```

Visualmente:

```
✉  harry@email.com
```

Su árbol es:

```
Row
├── Icon
└── Text
```

1.14 PODEMOS ANIDAR WIDGETS

Los widgets pueden contener otros widgets y estos, a su vez, contener otros.

Por ejemplo:

```
Column(
```

```
  children: [
    Icon(Icons.person),
    Text('Harry Potter'),
    Text('Estudiante de DAM'),
    Row(
      children: [
        Icon(Icons.email),
        Text('harry@email.com'),
      ],
    ),
    ElevatedButton(
      onPressed: () {},
      child: Text('VER PERFIL'),
    ),
  ],
)
```

Su árbol de widgets es:

```
Column
├── Icon
├── Text
├── Text
├── Row
│   ├── Icon
│   └── Text
└── ElevatedButton
    └── Text
```

Observa que:

- `Column` utiliza `children` porque tiene varios hijos.
- `Row` también utiliza `children` porque tiene varios hijos.
- `ElevatedButton` utiliza `child` porque tiene un único hijo: `Text`.

> **Una interfaz Flutter se construye componiendo widgets: colocamos widgets dentro de otros widgets hasta formar la estructura que necesitamos.**

1.15 ALGUNOS WIDGETS BÁSICOS

Ya conocemos `Row` y `Column`. Veamos ahora algunos widgets sencillos que podemos utilizar para construir nuestras primeras interfaces.

`Text`

Permite mostrar texto:

```
Text('Harry Potter')
```

Icon

Permite mostrar un icono:

```
Icon(Icons.school)
```

También podemos modificar algunas de sus propiedades:

```
Icon(
  Icons.school,
  size: 60,
)
```

Image

Permite mostrar una imagen. Por ejemplo, podemos cargar una imagen disponible en Internet:

```
Image.network(
  'https://...',
)
```

Más adelante aprenderemos a utilizar imágenes almacenadas dentro del propio proyecto.

ElevatedButton

Permite crear un botón:

```
ElevatedButton(
  onPressed: () {},
  child: Text('ACEPTAR'),
)
```

Por ahora no nos preocupa qué ocurre al pulsarlo. Nos interesa reconocer su estructura:

```
ElevatedButton
└── Text
```

IconButton

Un botón también puede estar formado únicamente por un icono:

```
IconButton(
  onPressed: () {},
  icon: Icon(Icons.favorite),
)
```

En este caso utilizamos la propiedad `icon` para indicar qué icono mostrará el botón.

CircleAvatar

`CircleAvatar` permite mostrar contenido dentro de un círculo. Es muy habitual para representar usuarios o perfiles.

```
CircleAvatar(
  child: Icon(Icons.person),
)
```

Su estructura es:

```
CircleAvatar
└── Icon
```

De nuevo encontramos `child`, porque `CircleAvatar` contiene **un único hijo**.

`Divider`

`Divider` dibuja una línea que podemos utilizar para **separar visualmente contenidos**:

```
Divider()
```

Este widget es interesante porque, a diferencia de los anteriores, **no necesita contener ningún otro widget**.

Por ejemplo:

```
Column(
  children: [
    Text('Datos personales'),
    Divider(),
    Text('Datos de contacto'),
  ],
)
```

`Card`

`Card` permite **agrupar visualmente contenido relacionado**:

```
Card(
  child: Text('Harry Potter'),
)
```

Pero también podemos colocar dentro de ella una estructura completa:

```
Card(
  child: Column(
    children: [
      Icon(Icons.person),
      Text('Harry Potter'),
      Text('Estudiante de DAM'),
```

```
    ],
  ),
)
```

Fíjate:

```
Card
|
└── Column
    ├── Icon
    ├── Text
    └── Text
```

Aunque visualmente aparecen tres elementos dentro de la tarjeta, `Card` solo tiene **un hijo directo**: `Column`.

Por eso utiliza `child` y no `children`.

1.16 `ListTile`: información organizada en una fila

Flutter incluye widgets que ya combinan varios elementos habituales de una interfaz.

Uno de ellos es `ListTile`.

Por ejemplo:

```
ListTile(
  leading: Icon(Icons.email),
  title: Text('Correo electrónico'),
  subtitle: Text('harry@email.com'),
)
```

Obtendremos una estructura similar a:

```
✉    Correo electrónico
     harry@email.com
```

Sus propiedades nos permiten indicar qué queremos colocar en diferentes zonas:

```
leading     → elemento situado al principio
title       → contenido principal
subtitle    → contenido secundario
```

Podemos combinar varios `ListTile`:

```
Column(
  children: [
    ListTile(
      leading: Icon(Icons.email),
```

```
      title: Text('Correo electrónico'),
      subtitle: Text('harry@email.com'),
    ),
    Divider(),
    ListTile(
      leading: Icon(Icons.phone),
      title: Text('Teléfono'),
      subtitle: Text('600 123 456'),
    ),
  ],
)
```

Ya podemos construir interfaces bastante más cercanas a las de una aplicación real.

1.17 CONSTRUIR UNA INTERFAZ: PRIMERO PENSAMOS EN SU ESTRUCTURA

Supongamos que queremos construir una pantalla de perfil:

```
          👤

      Laura Pérez
   Estudiante de DAM

____________________

✉  laura@email.com

☎  600 123 456

     [ VER PERFIL ]
```

Antes de empezar a escribir código debemos preguntarnos:

¿Qué widgets aparecen?

Podemos identificar:

```
CircleAvatar
Text
Text
Divider
ListTile
ListTile
ElevatedButton
```

Después debemos preguntarnos:

¿Cómo están organizados?

Todos aparecen uno debajo de otro, por lo que necesitamos un `Column`:

```
Column
├── CircleAvatar
│   └── Icon
├── Text
├── Text
├── Divider
├── ListTile
├── ListTile
└── ElevatedButton
    └── Text
```

Y solo entonces escribimos el código.

> **No empieces pensando en código. Primero identifica los widgets y después piensa cómo deben combinarse.**

1.18 LO QUE HEMOS APRENDIDO

Al finalizar esta sesión deberías ser capaz de:

- reconocer algunos de los widgets más habituales de Flutter;
- distinguir entre `child` y `children`;
- utilizar `Row` y `Column` para combinar widgets;
- anidar unos widgets dentro de otros;
- interpretar un árbol de widgets;
- pasar de una interfaz sencilla a su árbol de widgets;
- y transformar ese árbol en código Flutter.

La idea fundamental de esta sesión es:

```
INTERFAZ
    ↓
¿QUÉ WIDGETS NECESITO?
    ↓
¿CÓMO SE RELACIONAN?
    ↓
ÁRBOL DE WIDGETS
    ↓
CÓDIGO FLUTTER
```

1.19 SESIÓN 4

1.20 WIDGETS BÁSICOS Y COMPOSICIÓN DE INTERFACES

1.20.1 CONSTRUIMOS INTERFACES COMBINANDO WIDGETS

En la sesión anterior aprendimos a reconocer la estructura básica de una aplicación Flutter. Ahora vamos a centrarnos en los elementos con los que construimos su interfaz.

En Flutter, una interfaz se construye **combinando widgets**. Algunos widgets muestran contenido, otros permiten realizar acciones y otros nos ayudan a agrupar y organizar los elementos.

Durante esta sesión aprenderemos a seleccionar los widgets adecuados y a combinarlos formando estructuras cada vez más complejas.

Ya hemos visto que en Flutter una interfaz está formada por **widgets organizados en forma de árbol**.

Ahora vamos a empezar a construir árboles más complejos. Para ello necesitamos distinguir dos situaciones:

- Un widget contiene **un único widget**.
- Un widget contiene **varios widgets**.

Flutter utiliza principalmente las propiedades `child` y `children` para expresar esta relación.

1.20.1.1 `child`: UN ÚNICO WIDGET HIJO

Algunos widgets contienen **un solo widget**. En estos casos utilizamos `child`.

Por ejemplo:

```
Center(
  child: Text('Hola Flutter'),
)
```

Su árbol es:

```
Center
  |
  └── Text
```

`Center` tiene un único hijo: `Text`.

Otro ejemplo:

```
Padding(
  padding: EdgeInsets.all(20),
```

```
  child: Text('Harry Potter'),
)
```

```
Padding
  │
  └── Text
```

La idea importante sería:

> **`child` → un widget hijo.**

1.20.1.2 `children`: VARIOS WIDGETS HIJOS

Otros widgets están diseñados para contener **varios widgets**.

Es el caso de `Column` y `Row`.

```
Column(
  children: [
    Icon(Icons.person),
    Text('Harry Potter'),
    Text('harry@email.com'),
  ],
)
```

Los corchetes [ ] indican que estamos proporcionando **una lista de widgets**.

Aquí `Column` contiene tres widgets.

```
Column
├── Icon
├── Text
└── Text
```

Por eso utilizamos:

```
children: [
  ...
]
```

La regla que hay que recordar es la siguiente:

```
child      →  UN widget

children   →  VARIOS widgets
```

1.20.1.3 `Column`: COLOCAR WIDGETS VERTICALMENTE

`Column` organiza sus hijos **de arriba hacia abajo**.

```
Column(
  children: [
```

```
    Icon(Icons.person),
    Text('Harry Potter'),
    Text('harry@email.com'),
  ],
)
```

Visualmente:

```
      👤
   Harry Potter
harry@email.com
```

Y estructuralmente:

```
Column
├── Icon
├── Text
└── Text
```

> El aspecto visual y el árbol de widgets son dos formas diferentes de representar la misma interfaz.

1.20.1.4 `Row`: COLOCAR WIDGETS HORIZONTALMENTE

`Row` funciona de manera parecida, pero coloca sus hijos **de izquierda a derecha**.

```
Row(
  children: [
    Icon(Icons.phone),
    Text('Llamar'),
  ],
)
```

Resultado aproximado:

```
☎  Llamar
```

Árbol:

```
Row
├── Icon
└── Text
```

1.20.1.5 PODEMOS ANIDAR WIDGETS

Un `Column` puede contener un `Row`, y ese `Row` puede contener otros widgets:

```
Column(
  children: [
    Icon(Icons.person),
```

```
    Text('Harry Potter'),
    Text('harry@email.com'),
    Row(
      children: [
        ElevatedButton(
          onPressed: () {},
          child: Text('EDITAR'),
        ),
        ElevatedButton(
          onPressed: () {},
          child: Text('SALIR'),
        ),
      ],
    ),
  ],
)
```

El árbol sería:

```
Column
├── Icon
├── Text
├── Text
└── Row
    ├── ElevatedButton
    │   └── Text
    └── ElevatedButton
        └── Text
```

Y aquí ya aparece algo interesante: **`ElevatedButton` utiliza `child` porque contiene un único widget**, mientras que `Row` utiliza `children` porque contiene varios.

1.20.2 CINCO WIDGETS BÁSICOS QUE VAMOS A UTILIZAR

No les metería todavía veinte widgets. Para estas horas trabajaría esencialmente con estos:

Widget	Para qué sirve
`Text`	Mostrar texto
`Icon`	Mostrar un icono
`Image`	Mostrar una imagen
`ElevatedButton`	Mostrar un botón
`Column`	Organizar widgets verticalmente
`Row`	Organizar widgets horizontalmente
Center	Centrar un widget

1.20.3 Cómo se usan

```
Text('Harry Potter')
Icon(Icons.school)
Image.network(              //Permite mostrar una imagen obtenida de Internet
  'https://...',
)
ElevatedButton(
  onPressed: () {},
  child: Text('ACEPTAR'),
)
```

Para centrar los widgets dentro de una columna y dentro de una fila se utiliza su propiedad :

```
mainAxisAlignment: MainAxisAlignment.center
```

Esta línea indica que los elementos se colocan **centrados en el eje principal**.

- En un `Column` → el eje principal es **vertical** ↕, así que los centra verticalmente.
- En un `Row` → el eje principal es **horizontal** ↔, así que los centra horizontalmente.

1.21 SESIÓN 5

1.22 LAYOUT Y DISTRIBUCIÓN DE WIDGETS

Hasta ahora hemos aprendido a construir interfaces combinando widgets:

```
Card
└── Column
    ├── Text
    ├── Text
    └── Row
        ├── IconButton
        └── IconButton
```

Pero todavía nos falta responder a preguntas como:

- ¿Cómo centro un elemento?
- ¿Cómo separo dos widgets?
- ¿Cómo hago que un elemento ocupe el espacio disponible?
- ¿Cómo distribuyo varios elementos por toda la pantalla?
- ¿Cómo coloco un widget encima de otro?

En Flutter, **el layout también se construye con widgets**. Algunos widgets muestran contenido (`Text`, `Image`, `Icon`), mientras que otros sirven fundamentalmente para **organizar, dimensionar o posicionar otros widgets**.

1.22.1 ROW Y COLUMN: DOS DIRECCIONES

Ya conocemos:

```
Row(
  children: [
    Icon(Icons.home),
    Text('Inicio'),
  ],
)
```

y:

```
Column(
  children: [
    Text('Nombre'),
    Text('Correo'),
  ],
)
```

La diferencia fundamental es la dirección en la que colocan sus hijos:

```
ROW

[ A ]  [ B ]  [ C ]

————————————→
  eje principal


COLUMN

[ A ]
  |
[ B ]    eje principal
  |
[ C ]
  ↓
```

Pero todo `Row` y `Column` tiene en realidad **dos ejes**.

En un `Row`

```
            eje transversal
                  ↓

      [ A ]   [ B ]   [ C ]

      ——————————————————→
            eje principal
```

El eje principal es **horizontal** y el transversal es **vertical**.

En un `Column`

```
          eje transversal →

              [ A ]
                |
              [ B ]
                |
              [ C ]
                ↓
          eje principal
```

El eje principal es **vertical** y el transversal es **horizontal**.

Esta idea será importante porque Flutter utiliza dos propiedades:

```
mainAxisAlignment   → eje principal
crossAxisAlignment  → eje transversal
```

1.23 `MAINAXISALIGNMENT`: DISTRIBUIR POR EL EJE PRINCIPAL

Supongamos:

```
Row(
  children: [
    Icon(Icons.home),
    Icon(Icons.search),
    Icon(Icons.settings),
  ],
)
```

Los tres widgets aparecen juntos al principio.

Podemos decidir cómo distribuirlos:

```
Row(
  mainAxisAlignment: MainAxisAlignment.center,
  children: [
    Icon(Icons.home),
    Icon(Icons.search),
    Icon(Icons.settings),
  ],
)
```

Algunos valores importantes son:

```
start
[■][■][■]-------------------
```

```
center
----------[■][■][■]----------


end
------------------[■][■][■]


spaceBetween
[■]----------[■]----------[■]


spaceAround
---[■]------[■]------[■]---


spaceEvenly
-----[■]-----[■]-----[■]-----
```

> `mainAxisAlignment` controla cómo se distribuyen los hijos en el **eje principal**.

Y cuidado: el eje principal cambia.

```
Row       → horizontal
Column    → vertical
```

Por tanto, la misma propiedad puede estar controlando horizontal o verticalmente dependiendo del widget.

1.24 `CROSSAXISALIGNMENT`: ALINEAR POR EL EJE TRANSVERSAL

La propiedad `crossAxisAlignment` permite controlar cómo se colocan los hijos de un `Row` o una `Column` en el **eje transversal**.

```
Column(
  crossAxisAlignment: CrossAxisAlignment.start,
  children: [
    Text('Laura Pérez'),
    Text('2.º DAM'),
    Text('Desarrollo de Interfaces'),
  ],
)
```

En un `Column`, el eje transversal es horizontal. Por eso `start` hace que los elementos se sitúen hacia el comienzo horizontal.

La regla que deben recordar es:

	`mainAxisAlignment`	`crossAxisAlignment`
`Row`	Horizontal	Vertical
`Column`	Vertical	Horizontal

Valor	Comportamiento
`CrossAxisAlignment.start`	Alinea los widgets al inicio del eje transversal.
`CrossAxisAlignment.center`	Centra los widgets en el eje transversal.
`CrossAxisAlignment.end`	Alinea los widgets al final del eje transversal.
`CrossAxisAlignment.stretch`	Extiende los widgets para ocupar todo el espacio disponible en el eje transversal.

1.25 ¿POR QUÉ ALGUNOS WIDGETS TIENEN `CHILD`, OTROS `CHILDREN` Y OTROS NINGUNO?

Ahora podemos entender algo que hemos ido viendo desde el principio.

Hay widgets cuya función consiste en trabajar con **un único widget**.

Por ejemplo:

```
Center(
  child: Text('Hola'),
)
```

`Center` recibe un widget y lo centra.

Por eso utiliza:

```
child → un hijo
```

Lo mismo sucede con:

```
Padding(
  child: ...
)

Expanded(
  child: ...
)

Card(
  child: ...
)
```

Otros widgets están diseñados para organizar **varios widgets**:

```
Row(
  children: [
    Text('Uno'),
    Text('Dos'),
    Text('Tres'),
  ],
)
```

Por eso utilizan:

```
children → varios hijos
```

Es el caso de:

```
Row
Column
Stack
Wrap
```

La documentación de Flutter distingue precisamente entre widgets de layout de un solo hijo y de múltiples hijos.

¿Y `Text`, `Icon` o `Image`?

```
Text('Hola')
```

`Text` no está diseñado para organizar otro widget: **él mismo representa contenido**.

Por eso no necesita:

```
child:
```

ni:

```
children:
```

Una buena regla mental sería:

> **`child` → un hueco para un widget.**
> **`children` → varios widgets.**
> **Propiedades como `title`, `icon`, `leading`... → huecos con una función específica.**

1.26 CENTER: CENTRAR UN WIDGET

`Center` es uno de los widgets de layout más sencillos:

```
Center(
  child: Text('Hola Flutter'),
)
```

Su función es colocar su `child` en el centro del espacio disponible.



1.27 3.6. PADDING: DEJAR ESPACIO ALREDEDOR

Observa:

```
Text('Hola Flutter')
```

Ahora:

```
Padding(
  padding: EdgeInsets.all(20),
  child: Text('Hola Flutter'),
)
```

`Padding` añade espacio alrededor de su hijo.



Podemos aplicar el mismo espacio a todos los lados:

```
padding: EdgeInsets.all(20)
```

Solo horizontal y vertical:

```
padding: EdgeInsets.symmetric(
  horizontal: 20,
  vertical: 10,
)
```

O elegir cada lado:

```
padding: EdgeInsets.only(
  left: 20,
  top: 10,
)
```

1.28 SIZEDBOX: TAMAÑO Y SEPARACIÓN

`SizedBox` permite reservar un espacio de tamaño determinado.

Por ejemplo:

```
Column(
  children: [
    Text('Nombre'),
    SizedBox(height: 20),
    Text('Correo'),
  ],
)
```

Visualmente:

```
Nombre


       ← 20 de separación


Correo
```

También puede establecer dimensiones:

```
SizedBox(
  width: 200,
  height: 50,
  child: ElevatedButton(
    onPressed: () {},
    child: Text('ENTRAR'),
  ),
)
```

Aquí el `SizedBox` tiene un `child` porque además de representar un espacio puede contener y restringir el tamaño de otro widget.

Código	Resultado
`SizedBox(height: 20)`	Reserva 20 píxeles de altura.

`SizedBox(width: 20)`	Reserva 20 píxeles de ancho.
`SizedBox(width: 200, height: 50)`	Establece ambas dimensiones.
`SizedBox(width: double.infinity)`	Solicita todo el ancho disponible.
`SizedBox(height: double.infinity)`	Solicita toda la altura disponible.

> **¿`CrossAxisAlignment.stretch` o `SizedBox(width: double.infinity)`?**
>
> En una `Column`, `CrossAxisAlignment.stretch` permite extender **todos sus hijos** horizontalmente. En cambio, `SizedBox(width: double.infinity)` permite solicitar que **un widget concreto** ocupe todo el ancho disponible.
>
> Si queremos extender todos los elementos, normalmente preferiremos `stretch`. Si queremos extender solamente uno, podemos utilizar `SizedBox`.

1.29 3.8. EXPANDED: OCUPA EL ESPACIO DISPONIBLE

Consideremos:

```
Row(
  children: [
    Text('Nombre'),
    Text('Laura Pérez'),
  ],
)
```

Ahora queremos que la segunda parte **ocupe el espacio restante**.

```
Row(
  children: [
    Text('Nombre'),
    Expanded(
      child: Text('Laura Pérez'),
    ),
  ],
)
```

La idea de `Expanded` es:

> **Ocupa el espacio disponible dentro de un `Row`, `Column` o `Flex`.**

Flutter lo utiliza precisamente para repartir el espacio restante entre los hijos.

Además, podemos tener varios:

```
Row(
  children: [
    Expanded(
      child: Text('A'),
```

```
    ),
    Expanded(
      child: Text('B'),
    ),
  ],
)
```

Los dos reciben una parte equivalente.

Y podemos utilizar `flex`:

```
Row(
  children: [
    Expanded(
      flex: 1,
      child: Text('A'),
    ),
    Expanded(
      flex: 2,
      child: Text('B'),
    ),
  ],
)
```

Entonces:

```
┌──────────┬────────────────────┐
│    A     │         B          │
└──────────┴────────────────────┘
     1               2
```

B recibe aproximadamente el doble de espacio que A.

1.30 SPACER: ESPACIO FLEXIBLE

Comparemos:

```
Row(
  children: [
    Text('Inicio'),
    SizedBox(width: 30),
    Text('Fin'),
  ],
)
```

con:

```
Row(
  children: [
    Text('Inicio'),
```

```
    Spacer(),
    Text('Fin'),
  ],
)
```

En el primer caso:

> Quiero **30 unidades de espacio**.

En el segundo:

> Quiero utilizar **el espacio libre disponible**.

```
SizedBox

Inicio   ←30→   Fin----------------


Spacer

Inicio-------------------------Fin
       ← espacio restante →
```

Por tanto:

> **`SizedBox` = espacio fijo.**
> **`Spacer` = espacio flexible.**

Internamente, `Spacer` está pensado precisamente como una forma sencilla de introducir espacio flexible en un `Flex`, `Row` o `Column`.

1.31 STACK: COLOCAR WIDGETS UNOS ENCIMA DE OTROS

Hasta ahora nuestros widgets se colocaban:

```
Row       → uno al lado del otro

Column    → uno debajo del otro
```

Pero algunas interfaces necesitan **superponer elementos**.

Por ejemplo:

```
┌──────────────────────────────┐
│                          ♥   │
│                              │
│          IMAGEN              │
│                              │
│                              │
└──────────────────────────────┘
```

Podemos hacerlo con:

```
Stack(
  children: [
    Image.network('...'),
    Icon(Icons.favorite),
  ],
)
```

`Stack` recibe `children` porque puede superponer varios widgets. Flutter lo define precisamente como un layout para solapar hijos.

1.32 `Positioned`: COLOCAR UN ELEMENTO DENTRO DE UN `Stack`

Normalmente necesitaremos controlar dónde aparece el elemento superpuesto.

```
Stack(
  children: [
    Image.network('...'),
    Positioned(
      top: 10,
      right: 10,
      child: Icon(Icons.favorite),
    ),
  ],
)
```

Ahora estamos diciendo:

```
10 desde arriba
10 desde la derecha
```



Aquí también podemos aprovechar para preguntar:

¿Por qué `Stack` utiliza `children` pero `Positioned` utiliza `child`?

Porque `Stack` organiza **varios elementos**, mientras que cada `Positioned` posiciona **un elemento concreto**.

> **`Positioned.fill()`** es una variante de `Positioned` que permite extender un widget hasta ocupar toda la superficie de un `Stack`. Es especialmente útil para colocar imágenes o fondos sobre los que queremos superponer otros elementos.

1.33 WRAP: CUANDO LOS ELEMENTOS NO CABEN

Imaginemos unas etiquetas:

```
[HTML] [CSS] [Flutter] [Dart] [Python] [Java]
```

Con un `Row`, si no caben horizontalmente podemos tener problemas de desbordamiento.

`Wrap` permite continuar en otra línea:

```
[HTML] [CSS] [Flutter] [Dart]
[Python] [Java]
```

Por ejemplo:

```
Wrap(
  spacing: 8,
  runSpacing: 8,
  children: [
    Chip(label: Text('HTML')),
    Chip(label: Text('CSS')),
    Chip(label: Text('Flutter')),
    Chip(label: Text('Dart')),
  ],
)
```

`spacing` controla la separación entre elementos y `runSpacing` la separación entre las distintas líneas.

Flutter define `Wrap` como un widget que distribuye sus hijos en varias filas o columnas cuando es necesario.

1.34 CONTAINER: UN WIDGET MUY VERSÁTIL

`Container` es especial porque permite combinar varias operaciones habituales:

- tamaño;
- padding;
- margen;
- fondo;
- bordes;
- decoración;
- un `child`.

Por ejemplo:

```
Container(
  width: 250,
  padding: EdgeInsets.all(16),
  margin: EdgeInsets.all(10),
```

```
  child: Text('Desarrollo de Interfaces'),
)
```

La documentación lo describe como un widget de conveniencia que combina operaciones frecuentes de posicionamiento, tamaño y decoración.

Advertencia:

> **Que `Container` pueda hacer muchas cosas no significa que debamos utilizarlo para todo.**

Si solo queremos padding:

```
Padding(...)
```

Si solo queremos una separación fija:

```
SizedBox(...)
```

Si solo queremos centrar:

```
Center(...)
```

El código expresa mejor **qué queremos conseguir**.

1.35 CONSTRUIR UN LAYOUT: PENSAR ANTES DE PROGRAMAR

Antes de escribir Flutter, podemos mirar una interfaz y preguntarnos:

```
┌──────────────────────────────────┐
│                                  │
│       👤   Hermione Granger      │
│            Estudiante de DAM     │
│                                  │
│  ────────────────────────────    │
│                                  │
│  ✉  hermione@hogwarts.edu        │
│                                  │
│  [ VER PERFIL ]            ♥     │
│                                  │
└──────────────────────────────────┘
```

Primero identificamos la estructura:

```
¿Vertical u horizontal?
        ↓
¿Qué elementos están agrupados?
        ↓
¿Necesito separar elementos?
        ↓
```

```
¿Alguno debe ocupar el espacio libre?
        ↓
¿Hay elementos superpuestos?
```

Y después podemos imaginar el árbol:

```
Card
└── Padding
    └── Column
        ├── Row
        │   ├── CircleAvatar
        │   ├── SizedBox
        │   └── Column
        │       ├── Text
        │       └── Text
        │
        ├── Divider
        │
        ├── ListTile
        │
        └── Row
            ├── ElevatedButton
            ├── Spacer
            └── IconButton
```

1.36 FLUTTER INSPECTOR: VER EL ÁRBOL DE WIDGETS

Cuando las interfaces empiezan a crecer, mirar únicamente el código puede resultar complicado.

Flutter Inspector nos permite **inspeccionar la estructura de widgets de una aplicación mientras la estamos ejecutando**.

Una interfaz como:

```
Card(
  child: Padding(
    padding: EdgeInsets.all(16),
    child: Column(
      children: [
        Text('Laura Pérez'),
        Divider(),
        Row(
          children: [
            Icon(Icons.email),
            Text('laura@email.com'),
          ],
        ),
```

```
      ],
    ),
  ),
)
```

puede visualizarse conceptualmente como:

```
Card
└── Padding
    └── Column
        ├── Text
        ├── Divider
        └── Row
            ├── Icon
            └── Text
```

Esto nos ayuda a relacionar tres cosas:

```
INTERFAZ VISUAL
      ↕
ÁRBOL DE WIDGETS
      ↕
  CÓDIGO DART
```

Y esa relación es precisamente una de las ideas más importantes de esta UT.

1.36.1 UNA IDEA IMPORTANTE SOBRE EL LAYOUT DE FLUTTER

> **En Flutter, el widget padre condiciona el espacio que puede ocupar su hijo.**

Por eso no siempre basta con poner `width: 200` y esperar que ocurra exactamente lo que imaginamos. Flutter utiliza un sistema de restricciones en el que las restricciones pasan del padre al hijo, los tamaños se comunican hacia arriba y el padre determina la posición. Es una idea central del sistema de layout, pero merece una explicación más adelante cuando aparezcan los primeros problemas reales de tamaño u *overflow*.

1.37 SESIÓN 6

1.38 PROPIEDADES Y PERSONALIZACIÓN DE WIDGETS

Las **propiedades** permiten modificar el aspecto y algunas características de los widgets sin cambiar la estructura de la interfaz.

1.38.1 PROPIEDADES MÁS UTILIZADAS

Widget	Propiedades	Ejemplo
`Text`	`style`, `textAlign`	`textAlign: TextAlign.center`
`TextStyle`	`fontSize`, `fontWeight`, `color`	`fontSize: 24`
`Icon`	`size`, `color`	`size: 50`
`Image`	`width`, `height`, `fit`	`fit: BoxFit.cover`
`Card`	`color`, `elevation`, `shape`	`elevation: 6`
`Container`	`width`, `height`, `decoration`	`width: 250`
`ElevatedButton`	`style`	`ElevatedButton.styleFrom(...)`

1.38.2 TEXTO

```
Text(
  'Flutter',
  textAlign: TextAlign.center,
  style: TextStyle(
    fontSize: 24,
    fontWeight: FontWeight.bold,
    color: Colors.blue,
  ),
)
```

1.38.3 ICONOS

```
Icon(
  Icons.star,
  size: 50,
  color: Colors.orange,
)
```

1.38.4 IMÁGENES

```
Image.network(
  'https://...',
  width: 250,
  height: 150,
  fit: BoxFit.cover,
)
```

`BoxFit`	Resultado
`contain`	Muestra la imagen completa
`cover`	Cubre todo el espacio; puede recortar
`fill`	Rellena todo el espacio; puede deformar

1.38.5 BOTONES

```
ElevatedButton(
  onPressed: () {},
  style: ElevatedButton.styleFrom(
    backgroundColor: Colors.blue,
    foregroundColor: Colors.white,
    side: BorderSide(color: Colors.yellow, width: 1),
    padding: EdgeInsets.symmetric(horizontal: 12, vertical: 6),
    shape: RoundedRectangleBorder(
      borderRadius: BorderRadius.circular(8),
    ),
  ),
  child: Text('CONTINUAR'),
)
```

1.38.6 CARD

```
Card(
  color: Colors.blue.shade50,
  elevation: 6,
  shape: RoundedRectangleBorder(
    borderRadius: BorderRadius.circular(16),
  ),
  child: Text('Flutter'),
)
```

1.38.7 CONTAINER Y BOXDECORATION

`Container` resulta útil cuando necesitamos **personalizar una caja** combinando varias características.

```
Container(
  width: 250,
  height: 120,
  decoration: BoxDecoration(
    color: Colors.blue,
    borderRadius: BorderRadius.circular(16),
    shape: BoxShape.circle,
    border: Border.all(
      color: Colors.black,
      width: 2,
    ),
  ),
  child: Text('Flutter'),
)
```

1.38.8 CLIPRRECT

En Flutter, el widget ClipRRect (Clip Rounded Rectangle) permite recortar un widget y su contenido siguiendo un rectángulo con esquinas redondeadas.

Aunque se utiliza frecuentemente con imágenes, puede aplicarse a cualquier widget, como Container, Column, Text, etc.

```
ClipRRect(
  borderRadius: BorderRadius.circular(20),
  child: Image.network(
    'https://picsum.photos/400/200',
    width: 280,
    height: 140,
    fit: BoxFit.cover,
  ),
)
```

En este ejemplo, `ClipRRect` recorta las cuatro esquinas de la imagen con un radio de 20 píxeles.

1.38.9 COLORES

Colores predefinidos:

```
Colors.blue
Colors.red
Colors.orange
Colors.blue.shade100
```

Color hexadecimal:

```
Color(0xFF2C3E50)
#2C3E50  →  Color(0xFF2C3E50)  //Los dos primero FF representan opacidad
```

1.38.10 ¿QUÉ PROPIEDAD NECESITO?

Quiero modificar…	Utilizo…
Tamaño del texto	`fontSize`
Negrita	`fontWeight`
Color	`color`
Tamaño de un icono	`size`
Tamaño de una imagen	`width`, `height`
Ajuste de una imagen	`fit`
Fondo de un botón	`backgroundColor`
Texto/icono de un botón	`foregroundColor`
Sombra/elevación de Card	`elevation`

Esquinas redondeadas	`borderRadius`
Borde	`border`
Decoración de una caja	`decoration`

> **Recuerda:** no es necesario memorizar todas las propiedades. Lo importante es identificar **qué widget quieres modificar** y buscar **qué propiedad controla esa característica**.

1.39 SESIÓN 7

1.40 EVENTOS Y CALLBACKS

Los **eventos** permiten que la aplicación responda a las acciones del usuario.

```
USUARIO
   ↓
realiza una acción
   ↓
EVENTO
   ↓
CALLBACK
   ↓
se ejecuta código
```

1.40.1 ONPRESSED

Se utiliza al **pulsar un botón**.

```
ElevatedButton(
  onPressed: () {
    print('Botón pulsado');
  },
  child: Text('GUARDAR'),
)
```

```
Pulsar GUARDAR
      ↓
   onPressed
      ↓
    () { ... }   ← callback
      ↓
"Botón pulsado"
```

`onPressed` no proporciona ningún dato:

```
()
```

1.40.2 onChanged

Se ejecuta **cada vez que cambia un valor**.

En un `TextField`:

```
TextField(
  onChanged: (String texto) {
    print(texto);
  },
)
```

Si escribimos `Sol`:

```
S
So
Sol
```

1.40.2.1 ¿DE DÓNDE SALE TEXTO?

Flutter proporciona al callback el **nuevo contenido del `TextField`**:

```
Usuario escribe "Sol"
        ↓
      Flutter
        ↓
      "Sol"
        ↓
(String texto)
        ↓
 print(texto)
```

`texto` es un **parámetro**.

El nombre lo elegimos nosotros:

```
(String texto)
(String valor)
(String dato)
```

Los tres funcionarían igual.

> **Flutter decide qué dato proporciona. Nosotros elegimos el nombre del parámetro.**

1.40.3 ONSUBMITTED

Se ejecuta cuando el usuario **confirma el texto introducido**.

```
TextField(
  onSubmitted: (String texto) {
    print(texto);
  },
)
```

Diferencia:

Callback	¿Cuándo se ejecuta?
`onChanged`	Cada vez que cambia el texto
`onSubmitted`	Cuando se confirma el texto

1.40.4 ONTAP

Se ejecuta al **tocar o hacer clic sobre un elemento**.

```
ListTile(
  title: Text('Mi perfil'),
  onTap: () {
    print('Perfil seleccionado');
  },
)
```

No recibe datos:

```
()
```

1.40.5 CON PARÁMETROS / SIN PARÁMETROS

Sin datos

```
onPressed: () {
  ...
}
onTap: () {
  ...
}
```

Con datos

```
onChanged: (String texto) {
  ...
}
onSubmitted: (String texto) {
  ...
}
```

Callback	Recibe	Ejemplo
`onPressed`	Nada	`()`
`onTap`	Nada	`()`
`onChanged` de `TextField`	Nuevo texto	`(String texto)`
`onSubmitted` de `TextField`	Texto confirmado	`(String texto)`

Idea clave

```
EVENTO
  ↓
Flutter ejecuta
  ↓
CALLBACK
  ↓
puede proporcionar DATOS
```

> **Evento:** algo que ocurre.
> **Callback:** función que se ejecuta cuando ocurre.
> **Parámetro:** dato que Flutter puede proporcionar al callback.

Por ahora:

```
EVENTO → CALLBACK → print()
```

Más adelante:

```
EVENTO → CALLBACK → cambia la interfaz
                          ↓
                       ESTADO
```

1.41 GESTIÓN DEL ESTADO

Hasta ahora hemos creado prácticamente toda la interfaz dentro de `main()`:

```
void main() {
  runApp(
    MaterialApp(
      home: Scaffold(
        body: Text('Hola'),
      ),
    ),
  );
}
```

Esta forma de trabajar es útil para empezar, pero **no es habitual construir toda una aplicación dentro de `main()`**.

A medida que una aplicación crece, dividimos la interfaz en **widgets más pequeños y organizados**.

Por ejemplo:

```
MiApp
├── Cabecera
├── PerfilUsuario
├── Contenido
└── PiePagina
```

Flutter nos permite crear nuestros propios widgets. Para ello utilizaremos principalmente dos tipos **`StatelessWidget`** y **`StatefulWidget`**:

`StatelessWidget`	`StatefulWidget`
Su contenido no necesita cambiar durante su uso	Su contenido puede cambiar durante su uso
Ejemplo: una cabecera	Ejemplo: un panel con un contador

```
Widget
├── StatelessWidget  → interfaz sin cambios
└── StatefulWidget   → interfaz con cambios
```

1.41.1 STATELESSWIDGET

Un `StatelessWidget` nos permite crear un **widget propio cuyo contenido no necesita cambiar durante su uso**.

Por ejemplo, podemos crear una cabecera:

```
class Cabecera extends StatelessWidget {

  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        Icon(Icons.school),
        Text('Desarrollo de Interfaces'),
        Text('2.º DAM'),
      ],
    );
  }
}
```

Ahora podemos utilizarlo como cualquier otro widget:

```
body: Cabecera()
```

1.41.2 ¿QUÉ SIGNIFICA CADA PARTE?

Código	Significado
`class Cabecera`	Creamos una clase llamada `Cabecera`
`extends StatelessWidget`	`Cabecera` hereda de `StatelessWidget` y se convierte en un widget creado por nosotros
`@override`	Redefinimos un método heredado (el método build)
`build()`	Construye la interfaz del widget. Este métoddo indica a Flutter qué interfaz debe dibujar
`BuildContext context`	Información sobre la posición del widget en el árbol

`return`	Devuelve el árbol de widgets que se mostrará

1.41.3 ¿QUÉ OCURRE SI LA INTERFAZ TIENE QUE CAMBIAR?

Supongamos que queremos crear un contador:

```
Contador: 0

[ SUMAR ]
```

Al pulsar **SUMAR** queremos obtener:

```
Contador: 1
```

y después:

```
Contador: 2
```

Ahora aparece un concepto nuevo: **el estado**.

1.41.4 ESTADO

> **El estado es información que puede cambiar durante la ejecución y cuyo valor afecta a la interfaz.**

Ejemplos de información que cambia:

Estado	Tipo	Puede provocar...
`contador = 0`	`int`	Mostrar otro número
`favorito = false`	`bool`	♡ → ♥
`notificaciones = true`	`bool`	Activar/desactivar un `Switch`
`volumen = 50`	`double`	Cambiar un `Slider`

> ⚠ **No todas las variables son estado**

Una variable forma parte del estado de la interfaz cuando:

```
¿Puede cambiar?
      +
¿Ese cambio afecta a la interfaz?
      ↓
     ESTADO
```

La pregunta que podemos hacernos para saber si una variable es estado es:

> **Si cambia este dato, ¿debería cambiar algo en pantalla?**

1.41.5 STATEFULWIDGET

Cuando necesitamos **guardar un estado que puede cambiar y actualizar la interfaz**, utilizamos `StatefulWidget`.

Un `StatefulWidget` utiliza **dos clases**:

```
class Contador extends StatefulWidget {

  @override
  State<Contador> createState() {
    return _ContadorState();
  }
}
```

y:

```
class _ContadorState extends State<Contador> {

  int contador = 0;

  @override
  Widget build(BuildContext context) {
    return Text('$contador');
  }
}
```

1.41.6 ¿PARA QUÉ SIRVE CADA UNA?

`Contador` es **una clase creada por nosotros**, que hereda de `StatefulWidget`.

Esta clase sobreescribe el método `createState()` para **crear un objeto de la clase de estado asociada a nuestro widget** `_ContadorState`.

`_ContadorState` es una clase también creada por nosotros que hereda de State<Contador>.

Encontraremos muchas expresiones parecidas en Dart

```
State<Contador>     → State asociado a Contador
List<String>        → lista de String
List<int>           → lista de int
```

A este mecanismo se le llama **genéricos**. Por ahora no necesitamos conocerlo en profundidad; basta con entender que lo que aparece entre < > indica **el tipo concreto con el que trabaja esa clase**.

Por otro lado, el nombre _ContadorState empieza por un guion bajo para indicar que ese elemento es **privado dentro de su biblioteca**.

Finalmente `_ContadorState` sobreescribe el método `build()` que es donde se construye la intefaz con los datos del estado.

En resumen:

- `Contador` → define nuestro `StatefulWidget` y crea su objeto `State`.

- `_ContadorState` → guarda los datos que cambian y construye la interfaz utilizando esos datos.

Código	¿Qué significa?
`class Contador`	Clase que hemos creado nosotros
`extends StatefulWidget`	`Contador` hereda de `StatefulWidget`
`createState()`	Crea el objeto `State` asociado al widget
`_ContadorState`	Clase de estado que hemos creado nosotros
`_`	Indica que el elemento es privado dentro de su biblioteca
`extends State<Contador>`	`_ContadorState` hereda del `State` asociado a `Contador`
`<Contador>`	Indica que `State` trabaja con un widget de tipo `Contador`
`int contador = 0`	Dato que forma parte del estado
`build()`	Construye la interfaz utilizando el estado

1.41.7 SETSTATE( )

Cambiar el valor de una variable **no basta para actualizar la pantalla**.

Para indicar a Flutter que el estado ha cambiado utilizamos:

```
setState(() {
  contador++;
});
```

`setState()` provoca que Flutter vuelva a ejecutar `build()` para actualizar la interfaz.

```
USUARIO
   ↓
EVENTO
   ↓
CALLBACK
   ↓
setState()
   ↓
CAMBIA EL ESTADO
   ↓
build()
   ↓
INTERFAZ ACTUALIZADA
```

Ejemplo:

```
ElevatedButton(
  onPressed: () {
    setState(() {
```

```
      contador++;
    });
  },
  child: Text('Sumar'),
)
```

Al pulsar:

```
contador = 0
     ↓
   SUMAR
     ↓
setState()
     ↓
contador = 1
     ↓
build()
     ↓
se muestra 1
```

No escribas cosas como la siguiente:

```
contador++;
setState(() {});
```

Los datos del estado que provocan cambios en la interfaz deben modificarse dentro de setState() porque así dejas claro qué variables estamos modificando en esa actualización. Ahora solo tienes una variable, pero imagínate cuando tengas 10.

1.41.8 EJEMPLO COMPLETO

```
class Contador extends StatefulWidget {

  @override
  State<Contador> createState() {
    return _ContadorState();
  }
}

class _ContadorState extends State<Contador> {

  int contador = 0;

  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        Text('Contador: $contador'),

        ElevatedButton(
          onPressed: () {
```

```
            setState(() {
              contador++;
            });
          },
          child: Text('Sumar'),
        ),
      ],
    );
  }
}
```

Identifica las piezas:

```
StatefulWidget → Contador

State          → _ContadorState

Estado         → contador

Evento         → onPressed

Callback       → () { ... }

Actualización  → setState()

Interfaz       → build()
```

1.41.9 OTROS EJEMPLOS DE ESTADO

Switch

```
bool notificaciones = true;
Switch(
  value: notificaciones,
  onChanged: (bool nuevoValor) {
    setState(() {
      notificaciones = nuevoValor;
    });
  },
)
```

Slider

```
double volumen = 50;
Slider(
  value: volumen,
  min: 0,
  max: 100,
  onChanged: (double nuevoValor) {
    setState(() {
      volumen = nuevoValor;
    });
  },
)
```

Aunque cambie el tipo de dato o el widget, **el mecanismo siempre es el mismo**:

```
EVENTO → CALLBACK → setState() → ESTADO → build() → INTERFAZ
```

1.41.10 ¿STATELESSWIDGET O STATEFULWIDGET?

Si el widget...	Utilizamos
Solo muestra información	`StatelessWidget`
No mantiene estado propio cambiante	`StatelessWidget`
Mantiene un dato que cambia	`StatefulWidget`
Ese cambio debe reflejarse en pantalla	`StatefulWidget`

Tener un botón no significa necesitar un `StatefulWidget`.

La pregunta es:

> **¿Necesito guardar un dato que cambia y ese cambio debe reflejarse en la interfaz?**

```
          ¿La interfaz depende
          de un estado que cambia?
                  |
          ┌───────┴────────┐
          |                |
          NO               SÍ
          ↓                ↓
   StatelessWidget   StatefulWidget
```

1.42 SESIÓN 8

1.42.1 NAVEGACIÓN ENTRE PANTALLAS

A diferencia de lo que hemos visto hasta ahora, las aplicaciones no suelen tener una única pantalla. Por ejemplo, una aplicación puede mostrar una pantalla de inicio, un catálogo de productos, los detalles de un producto y una pantalla de configuración.

En este apartado aprenderemos a **navegar entre distintas pantallas de una aplicación Flutter**.

En Flutter, **una pantalla también es un widget**.

Por tanto, no necesitamos aprender un mecanismo nuevo para construir pantallas: podemos utilizar los mismos widgets que ya conocemos.

Habitualmente, cada pantalla se implementa mediante una clase que contiene un `Scaffold`.

Por ejemplo, aquí tienes dospantallas:

Pantalla de inicio

```
class PantallaInicio extends StatelessWidget {
  const PantallaInicio({super.key});
```

Pantalla de detalles

```
class PantallaDetalles extends StatelessWidget {
  const PantallaDetalles({super.key});

  @override
```

```
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Inicio'),
      ),
      body: const Center(
        child: Text('Esta es la pantalla principal'),
      ),
    );
  }
}
```

```
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Detalles'),
    ),
    body: const Center(
      child: Text('Esta es la pantalla de detalles'),
    ),
  );
 }
}
```

Tenemos dos pantallas independientes, pero todavía no hemos establecido ninguna forma de desplazarnos entre ellas.

Para hacerlo utilizaremos `Navigator`.

> **Recuerda:** una pantalla no necesita ser un `StatefulWidget` por el simple hecho de permitir navegar a otras pantallas. Si no gestiona información que cambia, puede seguir siendo un `StatelessWidget`.

1.42.2 ¿QUÉ ES NAVIGATOR?

`Navigator` es el componente de Flutter encargado de **gestionar la navegación entre las pantallas de una aplicación**.

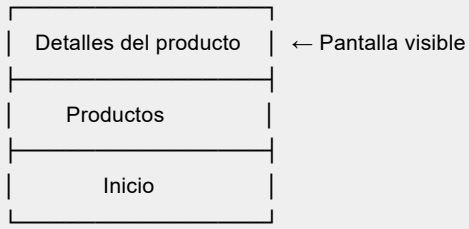
Para comprender su funcionamiento, debemos conocer el concepto de *pila* (*stack*).

1.42.2.1 LA PILA DE NAVEGACIÓN

Imagina que abrimos una aplicación y realizamos estas acciones:

1. Entramos en la pantalla de inicio.
2. Abrimos la pantalla de productos.
3. Seleccionamos un producto para consultar sus detalles.

Flutter puede representar este recorrido mediante una pila:



Cada vez que abrimos una nueva pantalla, se coloca encima de las anteriores.

Cuando regresamos, se retira la pantalla superior y vuelve a mostrarse la que estaba debajo.

Este comportamiento se conoce como **LIFO (*Last In, First Out*)**, es decir, el último elemento que entra es el primero que sale.

Flutter proporciona dos operaciones fundamentales para gestionar esta pila:

Operación	Función
`Navigator.push()`	Añade una nueva ruta a la pila de navegación.
`Navigator.pop()`	Retira la ruta superior de la pila.

Veamos cómo utilizarlas.

1.42.2.2 ABRIR UNA PANTALLA CON NAVIGATOR.PUSH()

Para abrir una nueva pantalla utilizamos el método `Navigator.push()`.

Su estructura habitual es:

```
Navigator.push(
  context,
  MaterialPageRoute(
    builder: (context) => const PantallaDetalles(),  (*)
  ),
);
```

¿Qué significa cada elemento?

Elemento	Función
Ruta (route)	Objeto que representa una entrada individual de navegación en Flutter. Define qué contenido se muestra y cómo se presenta al usuario.
`Navigator`	Widget que gestiona una pila de rutas, permitiendo añadir nuevas rutas mediante `push()` y retirar la ruta superior mediante `pop()`.
`context`	Permite localizar el `Navigator` dentro del árbol de widgets.
`MaterialPageRoute`	Clase que permite crear una ruta cuyo contenido se construye mediante una función `builder`, utilizando las transiciones propias de Material Design.
`builder`	Construye el widget que se mostrará en la nueva pantalla.
`PantallaDetalles()`	Es el widget que representa la pantalla de destino.

¿Por qué necesitamos localizar el Navigator? Porque una aplicación puede tener varios Navigator. Hay un Navigator por defecto, que crea MaterialApp, que es el que nosotros utilizaremos por ahora, pero podría haber más, de ahí que necesitemos saber en qué lugar del árbol estamos. Te pongo un ejemplo de árbol con tres Navigator (Navigator root creado por MaterialApp y Navigator A y B creados por el desarrollador). Nosotros por ahora solo utilizaremos el Navigator de MaterialApp:

```
MaterialApp
```



(*) Las funciones flecha (=>) en Dart. En Dart podemos escribir funciones sencillas utilizando el operador flecha (=>), también llamado *arrow syntax*.

Una **función flecha** es una forma abreviada de escribir una función cuyo cuerpo contiene una única expresión.

Por ejemplo, estas dos funciones son equivalentes:

Función tradicional:

```
int doble(int numero) {
  return numero * 2;
}
```

Función flecha:

```
int doble(int numero) => numero * 2;
```

En ambos casos, la función recibe un número y devuelve su doble.

El operador => sustituye a las llaves y a `return`, permitiendo escribir la función de forma más compacta.

Función tradicional:

```
MaterialPageRoute(
  builder: (context) {
    return const PantallaDetalles();
  },
)
```

Función flecha:

```
MaterialPageRoute(
  builder: (context) => const PantallaDetalles(),
)
```

Elemento	Significado
`builder`	Propiedad que recibe una función.
`(context)`	Parámetro que recibe la función.
`=>`	Indica que la función devuelve el resultado de la expresión situada a su derecha.
`const PantallaDetalles()`	Widget que devuelve la función.

Habitualmente, se utiliza dentro de un evento como `onPressed` de un botón:

```
ElevatedButton(
  onPressed: () {
    Navigator.push(
      context,
      MaterialPageRoute(
        builder: (context) => const PantallaDetalles(),
      ),
    );
  },
  child: const Text('Ver detalles'),
)
```

Al pulsar el botón, Flutter abre `PantallaDetalles`.

> **Recuerda:** podemos navegar a cualquier otra pantalla cambiando el widget indicado en `builder`. La navegación se ejecuta como respuesta a un evento, utilizando los *callbacks* que ya conocemos.

1.42.2.3 REGRESAR CON NAVIGATOR.POP()

Una vez abierta una pantalla, podemos regresar a la anterior mediante:

```
Navigator.pop(context);
```

Este método retira la ruta actual de la pila de navegación.

Por ejemplo, podemos utilizarlo dentro de un botón:

```
ElevatedButton(
  onPressed: () {
    Navigator.pop(context);
  },
  child: const Text('Volver'),
)
```

Cuando el usuario pulse «Volver», se cerrará la pantalla actual y aparecerá la anterior.

Por otro lado, cuando utilizamos un `Scaffold` con un `AppBar`, Flutter normalmente incorpora automáticamente una flecha de retroceso si la pantalla tiene una ruta anterior a la que regresar.

Por tanto, podemos volver de dos formas:

- Mediante la flecha de retroceso del `AppBar`.
- Mediante un botón que ejecute `Navigator.pop(context)`.

En ambos casos se retira la ruta actual de la pila.

Importante: `Navigator.pop()` no significa necesariamente «ir a Inicio». Significa regresar a la ruta anterior.

1.42.2.4 ABRIR CON NAVIGATOR.PUSHREPLACEMENT()

El método `Navigator.pushReplacement()` **sustituye la ruta actual por una nueva**.

A diferencia de `push()`, no añade simplemente otra pantalla a la pila, sino que reemplaza la que está en la parte superior.

Imagina que nuestra aplicación tiene estas pantallas:

Inicio → Identificación → Bienvenida

Si utilizamos `push()` para navegar desde Identificación hasta Bienvenida, la pila será:

```
┌────────────────────┐
│    Bienvenida      │ ← Pantalla actual
├────────────────────┤
│  Identificación    │
├────────────────────┤
│      Inicio        │
└────────────────────┘
```

Si pulsamos «Atrás», regresaremos a **Identificación**.

En cambio, si utilizamos `pushReplacement()`, la pila quedará así:

```
┌────────────────────┐
│    Bienvenida      │ ← Pantalla actual
├────────────────────┤
│      Inicio        │
└────────────────────┘
```

La pantalla **Identificación** ha sido sustituida por Bienvenida.

Por tanto, al pulsar «Atrás», regresaremos directamente a **Inicio**.

1.42.2.5 REGRESAR CON NAVIGATOR.POPUNTIL()

n ocasiones necesitamos **retroceder varias pantallas de una sola vez**.

Imagina que el usuario ha seguido este recorrido:

```
Inicio → Sala 1 → Sala 2 → Victoria
```

La pila de navegación contiene cuatro rutas:

```
┌──────────────────┐
│     Victoria     │  ← Pantalla visible
├──────────────────┤
│      Sala 2      │
├──────────────────┤
│      Sala 1      │
├──────────────────┤
```

```
|      Inicio     |
└─────────────────┘
```

Si ejecutamos `Navigator.pop(context)`, regresaremos a `Sala 2`. Para regresar directametne a Inicio, hay que ejecutar el método `Navigator.popUntil()`. El método `Navigator.popUntil()` permite retirar rutas de la pila **hasta que se cumpla una condición**. Se puede utilizar en, por ejemplo, un botón, siendo el botón de "Ir al inicio":

```
ElevatedButton(
  onPressed: () {
    Navigator.popUntil(
      context,
      (route) => route.isFirst,
    );
  },
  child: const Text('Volver al inicio'),
)
```

1.42.2.6 EJEMPLO NAVEGACIÓN ENTRE DOS PANTALLAS

```
import 'package:flutter/material.dart';

void main() {
  runApp(
    const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: PantallaInicio(),
    ),
  );
}


// ---------------------------------------------------
// PANTALLA DE INICIO
// ---------------------------------------------------


class PantallaInicio extends StatelessWidget {
  const PantallaInicio({super.key});


  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Inicio'),
      ),
      body: Center(
        child: ElevatedButton(
          onPressed: () {
            Navigator.push(
              context,
              MaterialPageRoute(
                builder: (context) => const PantallaDetalles(),
              ),
            );
          },
          child: const Text('Ver detalles'),
        ),
      ),
    );
```

```
  }
}

// ---------------------------------------------------
// PANTALLA DE DETALLES
// ---------------------------------------------------

class PantallaDetalles extends StatelessWidget {
  const PantallaDetalles({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Detalles'),
      ),
      body: Center(
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            const Text(
              '¡Bienvenido a la segunda pantalla!',
            ),
            const SizedBox(height: 20),
            ElevatedButton(
              onPressed: () {
                Navigator.pop(context);
              },
              child: const Text('Volver'),
            ),
          ],
        ),
      ),
    );
  }
}
```